jamk

Practical Training Report

Gangster Pūkeko Games: Neverance

Aapo Vanhainen

Project report
Practical Training (BIT), Kalle Raijonkari
Return date: 16.12.2025
Degree Programme: Business Information Technology

jamk | Jyväskylän ammattikorkeakoulu
University of Applied Sciences

Contents

Figures

1 Gangster Pūkeko Games

Gangster Pūkeko Games is upcoming indie company, ran by my classmates David & Dion. Their first project is a mobile game called Neverance. Neverance is solar-punk themed, narrative heavy game with turn-based combat and quick exploration loop for items and enemies. Their debut game project started few months before my arrival, and expected release date is somewhere early 2026.

2 My job descriptions

2.1 Length of the training

The practical training started on October 2nd and ended on December 19th. The total length of the training was 351 hours, totaling 13 credits. The time sheet of exact hours can be found from Appendix 1. The amount of hours go over the 351 but only those are counted for the credit's sake.

2.2 My duties

2.2.1 Ink Typing SFX system with integration to Unity

As the project started, the audio design was at its very early stages. Not much had been planned and the arrival of music was detrimental to the soundscape of the game. Since the team wanted the sound effects to compliment the music, I started working on something else instead. We discussed early developments of sound effects on the first weeks and came up with idea for the dialogue system. Ink was designed to be the dialogue tool for the game, where the characters' dialogue would appear on screen letter by letter on the text box, as seen in Figure 1.

Figure 1. Dialogue box example from OMORI.

Similarly, as in OMORI when the text appears, each letter makes a sound. This was the idea that the team wanted for Neverance too. I started off by creating new FMOD project, making 2D time-line event with multi-instrument, found placeholder key-clacking sounds, added them into the project, made small randomizer for volume and pitch and then imported the project to Unity.

After this, I imported the Ink asset, made three scripts that allows the letters on the ink document to appear on the UI element letter by letter, and make randomized keyboard clack sound each time one is heard (video in Appendix 2).

The three scripts were: TypingSfxController which plays the keyboard clacks as one-shots and avoids spam of the audio with limiting float value, TypewriterTMP which feeds characters into a TextMeshPro with given characters per second speed and reveals each character of that file and stops the feed when it should stop, finally InkDialogueController which loads the compiled Ink story and extracts the next line of text on demand. I provided documentation for the team how to reproduce my steps, how everything works and what is the purpose of the ink dialogue system, and it can be found from Appendix 3.

2.2.2 FMOD project with general and special sound effects, with integration to Unity

Before the practical training I had no experience with FMOD, or Ink for that matter, so creating these projects were very entertaining and teaching. After I got some practice with the previous assignment I knew how to handle FMOD better, and started to create new project with a bit more ambition. The current project has proper folder structure (Figure 2), Mixer with volume randomizer and VCA's for SFX and music which we'll discuss later (Figure 3) and placeholder sounds for 3 different enemies and items, few general and UI sounds, as well as sweetness system.

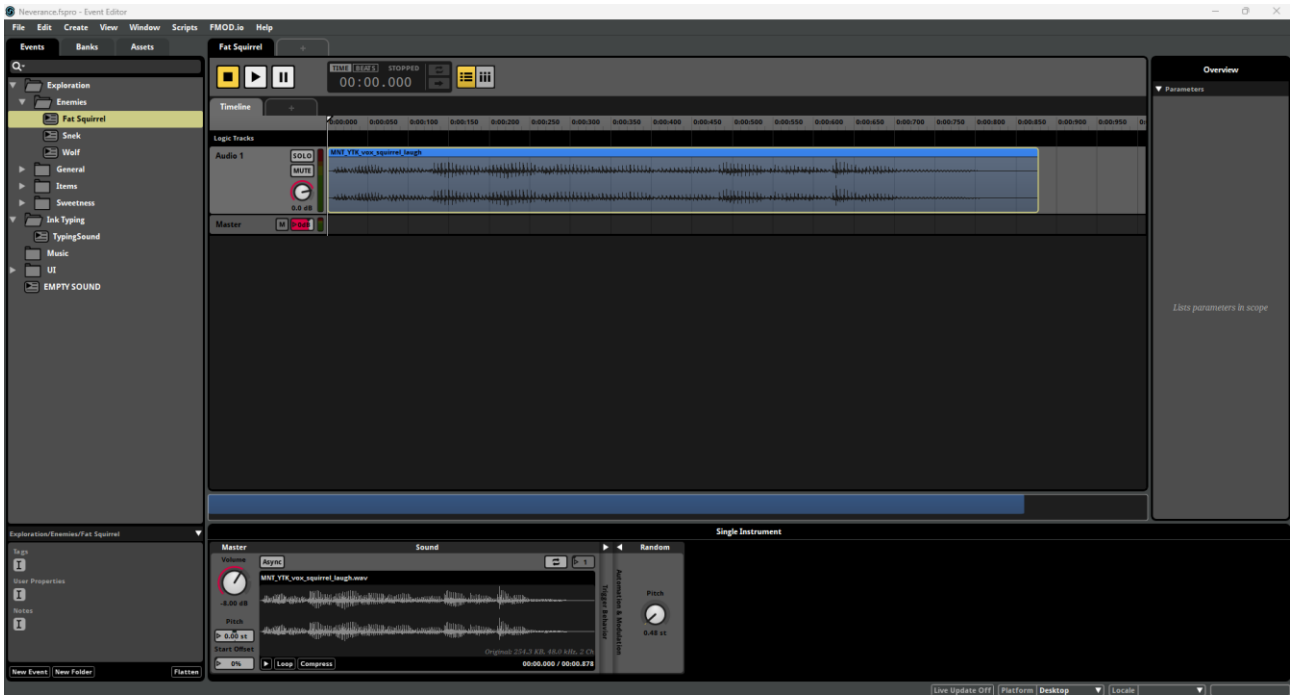

Figure 2. FMOD project folder structure

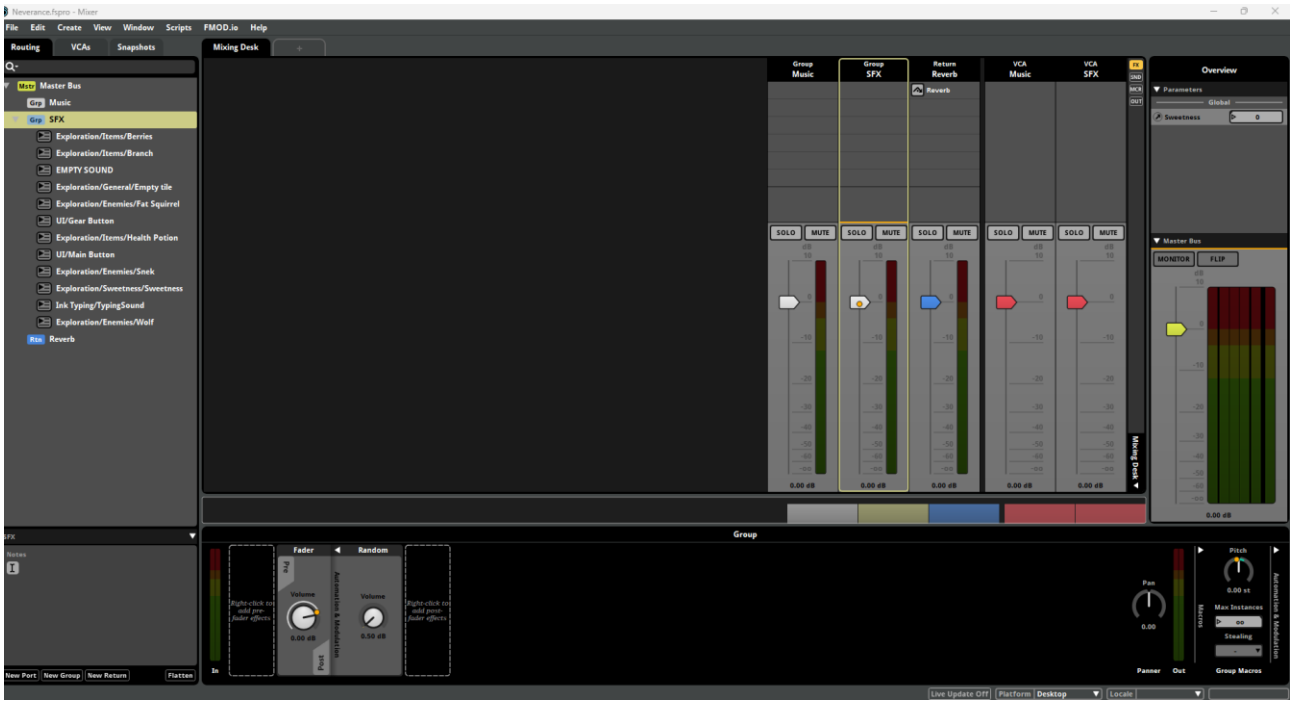

Figure 3. FMOD project's mixer with VCAs

The sound effect system is tied to a script in Unity, FMODAudioManager.cs, that has all set up in single script. It has volume sliders for quick iteration, all of the sound effects that are needed (or more that can be easily added) and the logic behind it. How the project and the script works, and how they can be implemented are found in the documentation I provided for the team, which is also found from Appendix 4.

The sweetness system is tied to the game's exploration phase where if player successfully finds same items in a pattern in a row, without failing by finding an empty tile or different item, there is a rising sweetness sound that plays from it. The idea was to create a piano medley which rises in tune, increasing and giving more excitement to the more items the player finds in a row. The sweetness system is created with Global Parameter in FMOD, that decides where the player is at in the sweetness system and where to go next, simple numerical value from 0 to 4. As the size of the patterns in explorations vary, from 1 to 5, the sweetness system adjusts correctly to play only the item sound when first found, then sweetness 1 and eventually when last item on the sequence is found it plays the sweetness max level of 4 (0-4 = 5). If the item is length of 1, it only plays the item sound, as these are legendary items, they already have that bling to it without the need of sweetness. If the length is 2, it plays no sweetness and on second it plays max sweetness sound.

The script allows for very easy to use methods to be used in other scripts to call for when audio needs to be played:

public void PlayPatternSound(string itemName) when specific pattern item sound needs to be played (e.g. Branch basic sound)

public void PlaySweetnessSounds(int sequenceCount) when in another script the sweetness is tracked and the int number can be fetched. Plays correct sweetness sound and resets if failed.

public void PlaySingleSound(EventReference sound) when a single shot of sound needs to be played such as clicking an UI element.

Everything is explained in detail in the documentation in Appendix 4.

2.3 Work Environment

I worked from home on my own PC, having Discord calls with team members when necessary. My schedule was around 10 am to 4 pm. I did drop by the office for meetings and GitHub merges, and sometimes to work. Me and Kimmo went to the studio (Äänipaja) once to record Foley sounds, but since there was no clear instructions or design as to what the sounds could be or even would be (“maybe we have crafting?” vibe) the recordings were fun, yet not that fruitful.

2.4 Needed skills and knowledge for the work

I received needed skills from tutorials and asking help from ChatGPT, as well some, but understandably narrow, guidance from the team. Since the audio portion of the game was on very early stages, it was acknowledged that the work provided for audio would be lackluster. My skills from

previous practical training with Conifer Digital on Versebound gave me insight into how to start creating sound effects and how to implement in Unity. The previous practical training helped a lot.

2.5 How could the work be improved

The work itself was mainly sound effects, FMOD and Ink based. The work could've been definitely be improved if there was a chance for me to do this training later, when audio was in better position current production-wise. Also the amount of work I needed for thesis (and other school stuff) ate away a lot of the hours of the practice, which was OK and discussed with David and Dion.

3 Learning in the workplace & personal challenges

I once again learned a lot with working with a small team. The slow down of the project after PGC and team's focus on school and thesis was challenging. The project itself didn't progress at all, or very slowly at best. Unfortunately a lot of the hours, as previously mentioned, were spent on thesis and school. But on the flipside, I learned a lot about FMOD and Ink and their integration to Unity. I'm much more confident on saying I can use FMOD, and it is surprisingly easy, thought it would be much more difficult. I didn't have time to create much sound effects, but I learned to implement MIDI keyboard and synthesizer into Reaper, allowing me to work with that in the future!

4 How BIT prepared me for the training

BIT helped me with knowledge gained for programming, using Unity, game design, audio design and editing, team working and social skills. Most notoriously the TiCorporate course, having such a wide range of tasks for an individual for a game project. In TiCorporate I worked on publishing, coding and production, all of the skills were very useful.

5 Quality and quantity of guidance from the degree program in relation to the work

As previously mentioned, the basic courses in design, programming, game engine, audio etc. were useful for hand-on skills for specific tasks but TiCorporate gave real-life experience, which helped a lot to be a trainee in a game company. During the practical training I did not ask for, nor needed, help from the degree program, since the team were able to help me perfectly.

6 Guidance and support from the workplace for performing work tasks

As mentioned, the actual time spent on the audio was not that much and the lack of audio design didn't support or guide it. All of this was of course understandable, but a bit frustrating. Having a office available was nice to catch up with others and solve problems together, especially design ones.

7 Company and work tasks vs suitability for student at BIT

On its current iteration it is eye-opening but hardly that suitable. The timing of taking people for practical training should not be close to graduation, unless the student has done most (including thesis) up to this point. The team is awesome and the project is interesting, would've hoped we had more time to focus on the actual game itself though.

8 Practical training seminar

We held a seminar together, discussing the topics about being a company lead as students in student environment. Unfortunately, I couldn't make it as I was too sick to get out of bed.

9 How would I develop the practical training process

I don't know so I'll leave the answer from previous report:

"I'm not sure. Some people might want to have more meetings with the teacher during the training, but I felt that it wasn't necessary. Finding a practical training spot is very difficult and requires a lot of effort and self-initiative, I hope that the students get a lot of help with that. Another fact is having a portfolio, we only have been suggested to do it, but having an actual course during third year spring would be great. Having an actual course for it would force students to make a portfolio but also help in making it look and sound good at every level would be so helpful. For example, I lack motivation to do it even though I know it is very important and would help me a lot."

10 Quality of practical training in relation to my studies and employment

As for studies it was 5/5, working in an indie company and making a game. For employment not that much (see answers above). As a practical training experience I would give it 3/5, very much fun but killed by the hurry of graduation and other school stuff.

Appendices

Appendix 1. Work schedule

Date	Day	Time In	Time Out	Notes about work	Midnight Mediation	Total Hours
29.9.2025	Monday					0:00
30.9.2025	Tuesday					0:00
1.10.2025	Wednesday					0:00
2.10.2025	Thursday	12:00 ip.	4:30 ip.			4:30
3.10.2025	Friday	10:00 ap.	4:00 ip.			6:00
4.10.2025	Saturday					0:00
5.10.2025	Sunday					0:00
				WEEKLY HOURS		10:30
6.10.2025	Monday	9:00 ap.	5:00 ip.			8:00
7.10.2025	Tuesday	8:00 ap.	4:00 ip.	PGC		8:00
8.10.2025	Wednesday	8:00 ap.	4:00 ip.	PGC		8:00
9.10.2025	Thursday	9:00 ap.	5:00 ip.			8:00
10.10.2025	Friday	9:00 ap.	5:00 ip.			8:00
11.10.2025	Saturday					0:00
12.10.2025	Sunday					0:00
				WEEKLY HOURS		40:00
13.10.2025	Monday	8:00 ap.	4:00 ip.			8:00
14.10.2025	Tuesday	8:00 ap.	4:00 ip.			8:00
15.10.2025	Wednesday	8:00 ap.	4:00 ip.			8:00
16.10.2025	Thursday	8:00 ap.	4:00 ip.			8:00
17.10.2025	Friday	8:00 ap.	4:00 ip.			8:00
18.10.2025	Saturday					0:00
19.10.2025	Sunday					0:00
				WEEKLY HOURS		40:00
20.10.2025	Monday	10:30 ap.	4:30 ip.			6:00
21.10.2025	Tuesday	9:30 ap.	6:30 ip.			9:00
22.10.2025	Wednesday	10:00 ap.	3:00 ip.		1:00	6:00
23.10.2025	Thursday	10:00 ap.	2:30 ip.			4:30
24.10.2025	Friday	10:30 ap.	5:00 ip.			6:30
25.10.2025	Saturday				4:30	4:30
26.10.2025	Sunday				4:30	4:30
				WEEKLY HOURS		41:00

Date	Day	Time In	Time Out	Notes about work	Midnight Mediation	Total Hours
27.10.2025	Monday	9:00 ap.	4:00 ip.			7:00
28.10.2025	Tuesday	10:00 ap.	16:00			6:00
29.10.2025	Wednesday	10:00 ap.	17:00			7:00
30.10.2025	Thursday	9:00 ap.	5:00 ip.			8:00
31.10.2025	Friday	10:00 ap.	5:00 ip.		1:00	8:00
1.11.2025	Saturday					0:00
2.11.2025	Sunday					0:00
				WEEKLY HOURS		36:00
3.11.2025	Monday	10:00 ap.	5:00 ip.			7:00
4.11.2025	Tuesday	9:30 ap.	4:00 ip.			6:30
5.11.2025	Wednesday	10:00 ap.	5:00 ip.			7:00
6.11.2025	Thursday	10:00 ap.	4:00 ip.			6:00
7.11.2025	Friday	10:00 ap.	4:00 ip.			6:00
8.11.2025	Saturday					0:00
9.11.2025	Sunday					0:00
				WEEKLY HOURS		32:30
10.11.2025	Monday	9:00 ap.	3:00 ip.			6:00
11.11.2025	Tuesday	10:00 ap.	4:30 ip.			6:30
12.11.2025	Wednesday	10:00 ap.	6:30 ip.			8:30
13.11.2025	Thursday	9:30 ap.	5:30 ip.			8:00
14.11.2025	Friday	11:00 ap.	12:00 ap.		4:30	17:30
15.11.2025	Saturday					0:00
16.11.2025	Sunday					0:00
				WEEKLY HOURS		46:30
17.11.2025	Monday	10:00 ap.	4:00 ip.			6:00
18.11.2025	Tuesday	10:30 ap.	5:00 ip.			6:30
19.11.2025	Wednesday	9:00 ap.	5:00 ip.			8:00
20.11.2025	Thursday	2:00 ip.	3:30 ip.		3:30	5:00
21.11.2025	Friday	10:30 ap.	4:30 ip.			6:00
22.11.2025	Saturday					0:00
23.11.2025	Sunday					0:00
				WEEKLY HOURS		31:30

Date	Day	Time In	Time Out	Notes about work	Midnight Mediation	Total Hours
24.11.2025	Monday	8:30 ap.	3:00 ip.			6:30
25.11.2025	Tuesday	10:30 ap.	4:30 ip.			6:00
26.11.2025	Wednesday	10:00 ap.	16:00			6:00
27.11.2025	Thursday	10:30 ap.	6:00 ip.			7:30
28.11.2025	Friday	10:00 ap.	3:00 ip.			5:00
29.11.2025	Saturday					0:00
30.11.2025	Sunday					0:00
				WEEKLY HOURS		31:00
1.12.2025	Monday	11:00 ap.	6:00 ip.			7:00
2.12.2025	Tuesday	11:00 ap.	5:00 ip.			6:00
3.12.2025	Wednesday	11:00 ap.	6:00 ip.			7:00
4.12.2025	Thursday	10:00 ap.	5:00 ip.			7:00
5.12.2025	Friday	11:00 ap.	6:00 ip.			7:00
6.12.2025	Saturday				5:00	5:00
7.12.2025	Sunday				6:00	6:00
				WEEKLY HOURS		45:00
8.12.2025	Monday	10:00 ap.	7:00 ip.			9:00
9.12.2025	Tuesday	10:00 ap.	6:00 ip.			8:00
10.12.2025	Wednesday	12:00 ip.	4:00 ip.			4:00
11.12.2025	Thursday	11:00 ap.	3:00 ip.			4:00
12.12.2025	Friday	10:00 ap.	1:00 ip.			3:00
13.12.2025	Saturday					0:00
14.12.2025	Sunday					0:00
				WEEKLY HOURS		28:00
15.12.2025	Monday	10:00 ap.	4:00 ip.			6:00
16.12.2025	Tuesday	10:00 ap.	1:00 ip.			3:00
17.12.2025	Wednesday					0:00
18.12.2025	Thursday					0:00
19.12.2025	Friday					0:00
20.12.2025	Saturday					0:00
21.12.2025	Sunday					0:00
				WEEKLY HOURS		9:00

Appendix 2. Ink Dialogue system in play and Sweetness sound system in play

Ink: https://youtu.be/qQia-GGBD30

Sweetness: https://youtu.be/Gn4zrYBVzoc

Appendix 3. Ink Dialogue system documentation

FMOD to Ink Integration

Dialogue system and clicking sound when text appear

How to recreate and use

By Aapo Vanhainen

Table of Contents

1. How the whole system works (big picture)

Setting up

Make sure you have the following: Unity project that has Assets: FMOD for Unity (https://assetstore.unity.com/packages/tools/audio/fmod-for-unity-2-02-161631), Ink for Unity (https://github.com/inkle/ink-unity-integration/releases) and TextMeshPro with TMP Essential Resources (in Unity: Window – TextMeshPro – Import TMP Essential Resources). Also have FMOD Studio installed with correct version, depending on the one you choose from Asset store, in this case 2.02.29 (https://www.fmod.com/download#fmodstudio).

Note: FMOD for Unity needs to be same version that your FMOD Studio, otherwise you cannot connect them together, aka it wont work.

When you have FMOD project integrated into Unity project, make sure from Unity top bar: FMOD – Edit Settings -> you double check under Bank Import that everything is correct:

Source type: Single Platform Build
Build path: leads to the folder with .bank files, in my case it is (C:\Users\aapov\Documents\FMOD Studio\Neverance\Build\Desktop).

Under Platform Specific choose Editor and make sure:

Live Update: Enabled.

FMOD (audio authoring)

You make one **2D event** for a single keyboard click using a **Multi Instrument** that contains several key-click samples. FMOD gives you natural variation (random sample, tiny pitch/volume randomization). You **Build Banks** (exports .bank files).

Unity (game runtime)

Unity loads the FMOD banks and can play FMOD events by name/reference.

Ink (story text)

Writers edit a simple text file (.ink). Unity's Ink integration compiles it to JSON that scripts can read at runtime.

Typewriter (UI)

A small script prints each line from Ink letter by letter into a TextMeshPro field. Every time a letter appears, we trigger one FMOD one-shot of the click event → sounds like typing.

Result: when the dialogue shows letters, you hear clicks; when it stops, clicks stop — automatically.

FMOD setup

Create a 2D event for per-character clicks

- Open FMOD Studio 2.02.29 (must match Unity integration).
- New Event → make sure it's 2D Timeline (no 3D/spatializer).
    - On left bar make sure you are under Events, from empty space right click – Event Defaults -> 2D Timeline
- Name the Event e.g. TypingDialogue (double click or Right Click -> Rename)
- On the track add a Multi Instrument.
    - Right click empty bar in the middle that is part of Audio 1, above Master
- Drag all your WAVs into the Multi's Playlist.
    - Left bar Assets, select all and drag and drop: on the bottom there is Playlist, an empty box
- In the Playlist header, choose Shuffle (should be on default).
- Right-click Pitch (text on the left side under Master) → Add Modulation → Random -> flip the knob to +0.32 - 0.48 semitones [st]).
- Right-click Volume → Add Modulation → Random (+0.5 dB). Same that you did to Pitch.
- Look under Conditions, right side of the box where you dropped the .WAVS and set (if not shown, open it with arrow)
    - Polyphony: 1–2
    - Stealing: Oldest.
- On the Left bar where Events is, go to Banks
    - Right Click -> New Bank -> name it SFX
- Go back to Events and Right click the Event you created e.g. TypingDialogue -> Assing to Bank -> Browse -> click your newly created SFX

Build banks

File → Build (F7). Ensure you get Master.bank, Master.strings.bank, and your SFX.bank. They should be in your FMOD set folder/Build/Desktop. If you don't know where to find them in FMOD go to Edit -> Preferences -> Build -> Build banks output directory (optional) -> it is blank by default and should be set so, now click Browse… -> that is the folder.

Unity project wiring

FMOD integration settings

These should be set up earlier but just to make sure.

- Unity menu: FMOD → Edit Settings.
- Source Type: FMOD Studio Project
- Bank Build Path: point to your FMOD Build/Desktop folder (the folder that contains the .bank files, not a single file).
- Load Banks: All.
- Check: Live Update on (port 9264).

Scene UI

- Create a Canvas.
- Add a TextMeshProUGUI (call it DialogueText) and style it.
- Add an empty GameObject DialogueSystem.

In our project there is already a Canvas and we don't want to break it, so just add Text – TextMeshPro. This is where the text in game is shown. DialogueSystem can be part of general gameobject that we use for systems e.g. GameManager but the following scripts need to be *some-where*.

Add scripts to DialogueSystem

- InkDialogueController (controls story flow).
- TypewriterTMP (reveals text slowly).
- TypingSfxController (plays click one-shots per character).

Assign references in Inspector

- In InkDialogueController:
    - Compiled Ink JSON: drag your compiled Ink asset (we'll create it in Part 4 unless it is already created).
    - Text UI: drag DialogueText (TMP component).
    - Typewriter: drag the TypewriterTMP component from DialogueSystem. (should be in same gameobject)
    - TypingSfx: drag the TypingSfxController component. (should be in same gameobject).
- In TypingSfxController:
    - Click Event: pick event: TypingDialogue (your FMOD event).
    - Min Interval: 0.02–0.05 (prevents spam if frame bursts happen).
        - Makes sure that there is *at least* min interval time between click sounds

Add a "Next" button

Create a UI Button → OnClick → assign InkDialogueController.NextLine().

This allows the text to continue. Could be different system later, e.g. automatically text continues or tapping anywhere makes the text continue (that could be big invisible UI button lol).

Ink basics (for writers)

Create your Ink file

- In Project: Create → Ink → Ink File; name it Dialogue.ink.
    - Right click in Assets (or in the folder you want this file to go to), this creates new file in your project
- Double-click Dialogue.ink to edit. Write your text on the black bar in the middle. Note make sure you open .ink file with the INK icon, not the normal text file it creates (Text Asset).
    - For me I couldn't press and write text on the black bar, instead I opened the file in Notepad and pasted the text there. After adding any text, I could edit in Unity. Try this if you can't write on the Unity black bar.

Write your lines

```
# typing_rate:12
Hello there! This is a typing test.

# typing_rate:8
Now it types slower...

# typing_rate:16
And now much faster!

    ➔ END
```

This was in my example text. Note there will be more and different texts in the game.

- The # typing_rate:X tag is optional — it is used to set characters per second in the typewriter (and you can also map it to FMOD if you later use the loop approach).

```
/// <summary>
/// Loads a new Ink story (you can call this from any other script).
/// </summary>
1 reference
public void LoadNewInkStory(TextAsset newInkJson)
{
    if (newInkJson == null)
    {
        Debug.LogWarning("InkDialogueController: Tried to load null Ink JSON.");
        story = null;
        textUI.text = "";
        return;
    }

    story = new Story(newInkJson.text);
    compiledInkJSON = newInkJson; // store reference
    NextLine();
}
```

LoadNewInkStory method allows replacing the variable compiledInkJSON in InkDialogueController, allowing correct text to be displayed. This method could be called in different script for example when loading in new scene or in scene start. Currently it starts immediately the text, if we want the text only appear after clicking somewhere or simply not start immediately when the file is loaded, then remove NextLine(); method at the end of this method.

Compile the Ink

- Select Dialogue.ink → click Recompile Ink (or it auto-compiles).
- You'll see a sibling file Dialogue.ink.json.
- This is what you drag into the InkDialogueController's "Compiled Ink JSON" field.

Using it

Press Play:

- The controller reads the first non-empty line from the compiled Ink story.
- TypewriterTMP prints it letter-by-letter.
- On each letter, TypingSfxController.ClickOneShot() plays your FMOD click.
- Press your Next button to show the next line.
    - Currently to make the text disappear, you need to press Next button after last text. A timer when it disappears and removing next button after showing last line could be added if wanted.

What each script does (plain English)

TypingSfxController

A very small FMOD wrapper with one method the typewriter calls per letter: ClickOneShot(). It plays a 2D one-shot event and rate-limits to avoid accidental bursts

TypewriterTMP

Feeds characters into a TextMeshPro label at a given speed (characters per second). It exposes two events:

- OnCharRevealed(char) — fires for each visible character.
- OnCompleted() — fires when the line finishes.

It also skips rich-text tags like … so tags don't slow the typing or trigger sounds.

InkDialogueController

Loads the compiled Ink story, extracts the next line of text on demand, reads tags like typing_rate:12 to configure speed, and starts the typewriter. It subscribes once to the typewriter's per-character callback and calls ClickOneShot() on every revealed letter. With Next button you can continue to next line, this is easiest currently to do and probably best because people read at different speeds.

[SerializeField] private TextAsset currentDialogueText;
This represents the lines of text that is displayed, whatever is written to that file. If you want it to play in start of the scene: drag and drop the one you want in the field, if you want to play it later: then use LoadNewInkStory method to add the one you want. It starts to play immediately after adding, with same next button logic.

Appendices

Appendix 1. InkDialogueController.cs

```
using UnityEngine;
using TMPro;
using Ink.Runtime;

public class InkDialogueController : MonoBehaviour
{
    /// <summary>
    /// Loads a compiled Ink story (.ink.json), feeds one line at a time into the typewriter,
    /// and wires per-character callbacks to the typing SFX.
    /// Supports a "typing_rate:X" tag on a line to control characters-per-second.
    /// </summary>

    [Header("References")]
    [Tooltip("Drag the compiled .ink.json asset here (generated by Ink integration) if you want it to play immediately on scene start. Or have it added later with LoadNewInkStory method")]
    [SerializeField] private TextAsset currentDialogueText;

    [SerializeField] private TMP_Text textUI;
    [SerializeField] private TypewriterTMP typewriter;
    [SerializeField] private TypingSfxController typingSfx;

    private Story story;
    private System.Action<char> _charHandler;

    void Awake()
    {
        // Subscribe to per-character typing sounds once
        _charHandler = _ => typingSfx.ClickOneShot();
```

```
    if (typewriter != null)
        typewriter.OnCharRevealed += _charHandler;
}

private void Start()
{
    // Only load if one is pre-assigned in Inspector
    if (currentDialogueText != null)
        LoadNewInkStory(currentDialogueText);
}
void OnDestroy()
{
    if (typewriter != null && _charHandler != null)
        typewriter.OnCharRevealed -= _charHandler;
}

/// <summary>
/// Loads a new Ink story (you can call this from any other script).
/// </summary>
public void LoadNewInkStory(TextAsset newInkJson)
{
    if (newInkJson == null)
    {
        Debug.LogWarning("InkDialogueController: Tried to load null Ink JSON.");
        story = null;
        textUI.text = "";
        return;
    }

    story = new Story(newInkJson.text);
```

```
    currentDialogueText = newInkJson; // store reference
    NextLine();
}

/// <summary>
/// Advance to the next line of text in the current story.
/// </summary>
public void NextLine()
{
    if (story == null)
    {
        Debug.LogWarning("InkDialogueController: No story loaded yet!");
        return;
    }

    string line = null;
    while (story.canContinue)
    {
        line = story.Continue().TrimEnd('\n', '\r');
        if (!string.IsNullOrWhiteSpace(line)) break;
    }

    if (string.IsNullOrWhiteSpace(line))
    {
        textUI.text = "";
        return;
    }

    // Look for a "typing_rate:X" tag to set cps for this line
    float cps = typewriter.DefaultCps;
```

```
        foreach (var tag in story.currentTags)
        {
            var idx = tag.IndexOf(':');
            var key = idx > 0 ? tag.Substring(0, idx).Trim() : tag.Trim();
            var val = idx > 0 ? tag.Substring(idx + 1).Trim() : null;

            if (string.Equals(key, "typing_rate", System.StringComparison.OrdinalIgnoreCase)
                && float.TryParse(val, out var v))
                cps = Mathf.Max(0f, v);
        }

        // Start typewriter animation; per-character clicks are fired by the callback
        typewriter.Reveal(line, cps);
    }
}
```

Appendix 2. TypeWriterTMP.cs

```
using System.Collections;
using UnityEngine;
using TMPro;

public class TypewriterTMP : MonoBehaviour
{
    /// <summary>
    /// Reveals text into a TMP field at a given characters-per-second speed.
    /// Skips over TMP rich-text tags (<b>, <i>, <color>, etc.) without delaying.
    /// Exposes events for per-character and completion.
    /// </summary>

    [Header("Target")]
    [Tooltip("The TMP text component that will receive characters.")]
    [SerializeField] private TMP_Text target;

    [Header("Speed")]
    [Tooltip("Default characters per second if typing rate is not defined in Ink text")]
    [SerializeField] private float defaultCps = 12f;
    [HideInInspector] public float DefaultCps => defaultCps;
    [Header("Sound Filtering")]
    [Tooltip("When true, whitespace characters won't trigger OnCharRevealed. Default: true")]
    [SerializeField] private bool skipWhitespaceSounds = true;

    /// <summary> Fires once per visible character (not for whitespace if filtering on). </summary>
    public System.Action<char> OnCharRevealed; // optional per-char callback

    /// <summary> Fires when the entire line has finished revealing. </summary>
    private Coroutine routine;
```

```
public System.Action OnCompleted; // add this

/// <summary> Start revealing the given text at the given cps. </summary>
public void Reveal(string text, float cps)
{
    if (target == null) return;
    if (routine != null) StopCoroutine(routine);
    routine = StartCoroutine(RevealRoutine(text, cps <= 0f ? defaultCps : cps));
}

/// <summary> Instantly set text (no animation). </summary>
public void SetInstant(string text)
{
    if (target == null) return;
    if (routine != null) StopCoroutine(routine);
    target.text = text;
}

private IEnumerator RevealRoutine(string text, float cps)
{
    target.text = "";
    int i = 0;
    float secPerChar = 1f / Mathf.Max(0.001f, cps);
    float t = 0f;

    while (i < text.Length)
    {
        t += Time.unscaledDeltaTime;
        while (t >= secPerChar && i < text.Length)
        {
```

```
            t -= secPerChar;

            if (text[i] == '<')
            {
                int close = text.IndexOf('>', i);
                if (close > i)
                {
                    target.text += text.Substring(i, close - i + 1);
                    i = close + 1;
                    continue;
                }
            }

            char c = text[i++];
            target.text += c;

            if (!(skipWhitespaceSounds && char.IsWhiteSpace(c)))
                OnCharRevealed?.Invoke(c);
        }
        yield return null;
    }

    OnCompleted?.Invoke(); // <-- notify end of line
  }



}
```

Appendix 3. TypingSfxController.cs

```
using UnityEngine;
using FMODUnity;

/// <summary>
/// Simple controller that plays a single FMOD one-shot each time
/// a character is revealed by the typewriter.
/// Designed for a 2D "typing click" event (a Multi Instrument with several samples).
/// </summary>
public class TypingSfxController : MonoBehaviour
{
    [Header("FMOD Settings")]
    [Tooltip("Assign your 2D typing click event (a Multi Instrument with several key samples). " +
        "This event should have Shuffle mode enabled and no looping.")]
    [SerializeField] private EventReference clickEvent;

    [Header("Rate Limiting")]
    [Tooltip("Minimum time (seconds) between clicks, to prevent rapid-fire spam. " +
        "Keep low, around 0.02–0.05 for natural pacing.")]
    [SerializeField] private float minInterval = 0.02f;

    // internal timing trackers
    private float lastClickTime;
    private int lastClickFrame = -1;

    /// <summary>
    /// Plays one FMOD one-shot typing sound, with built-in rate limiting.
    /// Call this once per visible character reveal.
    /// </summary>
    public void ClickOneShot()
```

```
    {
        // Skip if no event assigned
        if (clickEvent.IsNull)
            return;

        // Avoid triggering multiple clicks in the same frame
        if (lastClickFrame == Time.frameCount)
            return;

        // Avoid triggering faster than minInterval seconds
        if (Time.unscaledTime - lastClickTime < minInterval)
            return;

        // Play the 2D typing click one-shot
        RuntimeManager.PlayOneShot(clickEvent);

        // Record timing for throttling
        lastClickFrame = Time.frameCount;
        lastClickTime = Time.unscaledTime;
    }
}
```

Appendix 4. FMOD SFXs to Unity Integration documentatition

FMOD SFXs to Unity Integration

Sound effects in game

By Aapo Vanhainen

Table of Contents

Quick guide

Assuming everything is set up, if unclear then read further to the documentation.

- Create new 2D Timeline event in correct folder in FMOD project.
- Drag sound effect, add small (+-0.48 st) pitch variation.
- Save (CTRL+S) and Build (F7)
- Go to Unity, refresh banks if not automatically prompted
- Go to FMODAudioManager prefab
- add new sound effect
    - if General audio such as UI or general eploration audio:
        - Go to script FMODAudioManager.cs
        - add public EventReference yourSFX;
        - go to the script where you want to play the sound for example when enemy takes damage
        - Add the sound effect: FMODAudioManager.Instance.PlaySingleSound(FMODAudioManager.Instance.yourSFX);
            - *Note in this case the enemy takes damage method needs to find how it plays the correct sound (e.g. wolf take damage sound instead of snake take damage sound) itself. With references it can be done easily (for example DataContainer dataContainer.dataname and having the public EventReference name being the same name as dataname)*
    - if Pattern sound effect such as Resin:
        - in Inspector view of the gameobject FMODAudioManager find Patterns and click **+** icon to add new one
        - Assign its name (must be exactly as the scriptableobject's dataname (for example branch is **Branch** not **branch**))
        - Sound: find the correct sound that you just created (search function helps)
        - Max In Pattern: amount of items in the pattern, for example branch = 4
- Test and iterate. Remember: live update allows you to edit sounds in FMOD while in play mode in Unity without the need of saving and building every time.

Important!!
Each camera in every scene has to have FMOD audio listener instead of Unity one:

- From (main) camera, remove Audio listener component (in inspector)
- Add FMOD Studio Listener component
    - This allows sounds to be heard via FMOD instead Unity own audio system
    - Attentuation Object should be left blank, a 2D game shouldn't need spatialization

Setting up

Make sure you have the following: Unity project that has Assets: FMOD for Unity (https://assetstore.unity.com/packages/tools/audio/fmod-for-unity-2-02-161631). Also have FMOD Studio installed with correct version, depending on the one you choose from Asset store, in this case 2.02.29 (https://www.fmod.com/download#fmodstudio).
Note: FMOD for Unity needs to be same version that your FMOD Studio, otherwise you cannot connect them together, aka it wont work.

When you have FMOD project integrated into Unity project, make sure from Unity top bar: FMOD – Edit Settings -> you double check under Bank Import that everything is correct:

Source type: FMOD Studio Project
Build path: Assets/FMOD Studio/Neverance/Neverance.fspro (this should be same for everyone, test and report if doesn't work)

Under Platform Specific choose Editor and make sure:
Live Update: Enabled.
This allows you to update immediately in play mode, to test sounds etc.

Creating a sound in FMOD

- Open the FMOD project, go to Events on the left side.
- Select folder under which you want the sound effect to be added, for example Exploration/Items.
- Right click that folder and select Event Defaults -> 2D Timeline.
- Rename it, for example Berries.
- Right click the newly created event e.g. Berries -> Assign to Bank -> SFX
- Make sure your new event is selected -> on the Audio 1 bar on the Timeline (middle of the screen) right click -> Add single instrument
    - This is now your new sound effect under SFX sound bank.
- Move the blank box that you created to start (00:00). Click to select it if not yet selected. On the bottom of the screen a box titled Sound should've appeared. Under Master it has Volume knob, adjustable pitch and start offset.
- On the Sound box in the middle there is empty space. Drag your sound effect here.
- Since volume randomization is adjusted from Mixer, where pitch randomization is not possible, we need to do that now. NOTE: by selecting multiple events you can add random pitch to all of them at once, although it sometimes is a bit flaky and needs re-attempts but is possible.
- Make sure you have your new sound effect selected (clicking the instrument on the Timeline), look at the bottom box again titled Sound. On the left side of that there is Master and under that you can find Pitch, right click that -> Add modulation -> Random
- New box named Random appeared. Either manually turn the knob or double click "0.00 st" and set new value: 0.48 st (note this value might change and via testing the best is found, but for now that should be fine)
- Save (CTRL+S) and Build Banks (F7)

Unity

Unity loads the FMOD banks and can play FMOD events by name/reference. Anything that prompts, allow the FMOD to attempt fix. If you've changed folder names or item names the asset attempts to rename them and find new paths.

FMOD Audio Manager

Make sure the scene you are working on has the prefab FMODAudioManager (found Assets/Prefabs/FMODAudioManager). NOTE: when building, make sure this is only in the starting scene (mainmenu?) and not in other scenes, as it is a singleton.

NOTE! Every time there are changes made to the script, something might visually disappear. This is common visual bug of the inspector in Unity. For example Max In Pattern selection might disappear from the patterns altogether. This is fixed by clicking anything else on the Hierarchy or Project structure, making the inspector view change to show other item, then returning to FMODAudioManager prefab. Basically it refreshes the inspector view.

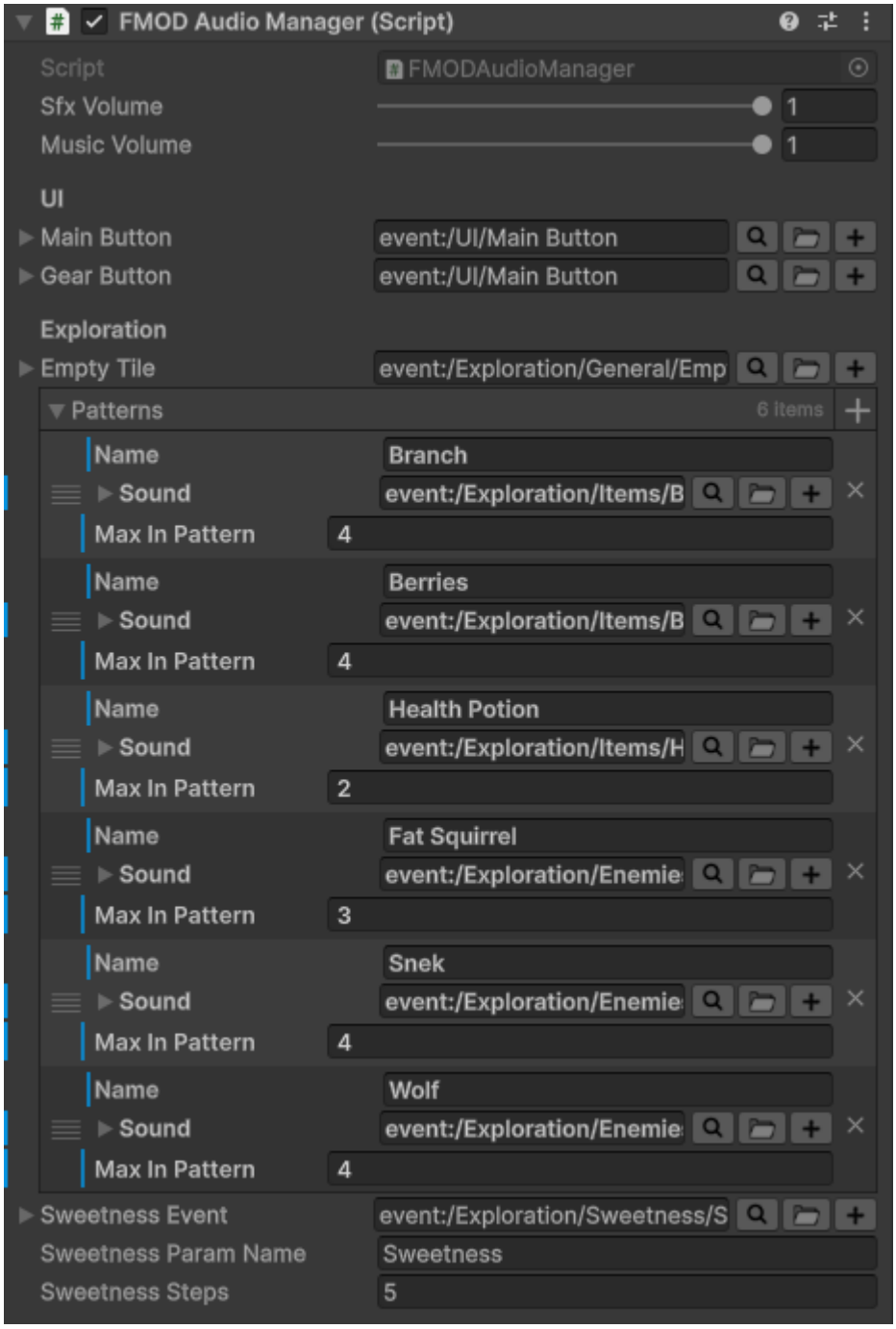

Figure 4: FMOD Audio Manager script.

Sliders

First two sliders are volume sliders for SFX and music. They are directly connected to FMOD Mixer's VCA's. Technical stuff in script, but in usage very simple: they allow in testing to adjust volume of music and sound effects for example if you want to mute/lower volume of music and just hear sound effects.

UI button sounds

Next is UI management. Currently Main Button and Gear Button. Add more in script under Header UI (for example public EventReference inventoryButton;). These are sound effects for UI elements, meant to play as single shots, very simple logic. Meant to be added on the method where the button action is, for example in mainmenu if there is button that leads to world map, there most likely is method (currently like this, might change):

```
public void OnWorldMapButtonClick()
{
    SceneManager.LoadScene(_worldMapScene);
}
```

So you would add sound effect here as follows:

```
public void OnWorldMapButtonClick()
{
    FMODAudioManager.Instance.PlaySingleSound(FMODAudioManager.Instance.mainButton);
    SceneManager.LoadScene(_worldMapScene);
}
```

This method allows to play one shot of the UI button sound made for all of the three main buttons in main menu (as designed, all 3 main menu buttons make same sound).

Exploration

General sound effects

At the top of Exploration there are general sound effects that are played as one shots for example in SlotClickHandler.cs when empty tile is found this line of code is added: FMODAudioManager.Instance.PlaySingleSound(FMODAudioManager.Instance.emptyTile);. More general sounds in the future could include flip a tile, run out of moves, gain new moves, etc.

Patterns

Patterns is every findable item in the exploration grid. Here you would add new ones when they are implemented. Click + icon on the right side of the top of it (directly right from the word Patterns, next to "6 items" in the Figure 1). Re-read NOTE in the beginning of chapter 4 Exploration!

- Add new pattern into to the script by clicking + icon.
- On the Name section write the EXACT name that is found from ScriptableObject's Data Name bar in Inspector. For example branch is **Branch** not **branch**.
    - Assets/ScriptableObjects/Items -> Branch
        - Data Name
            - This exact word, you can copypaste if you want
- Sound: find the correct sound for it
    - you can use the search by immediately starting to type which should speed up the work
- Max in Pattern: the size of the pattern. For example branch = 4
    - If unsure:
        - Assets/Prefabs/Exploration objects/ShapeUIObjects -> find the object for example BranchShape and calculate how many PatternImageBackground gameobjects there are
        - Also should be found from documentation (Neverance_GridPatterns or Design Document)

Sweetness

Sweetness is global parameter in FMOD. It is already set up to work regardless of pattern size. Only edit this if design change. The method in script plays always the sweetness level 4 (max) when last item is found in perfect sequence, regardless of pattern size. If the size is two it plays 0 then 4. If the size is one it plays ONLY 0 (purpose is to create the legendary item find sounds as perfect from get-go, so no sweetness is required to be added).

The Script:

```
using System.Collections.Generic;
using UnityEngine;
using FMODUnity;
using FMOD.Studio;

public class FMODAudioManager : MonoBehaviour
{
    public static FMODAudioManager Instance { get; private set; }

    [Header("FMOD Volume Settings")]
    private string sfxVcaPath = "vca:/SFX";
    [Range(0f, 1f)]
    [SerializeField] private float sfxVolume = 1f;

    private string musicVcaPath = "vca:/Music";   // make sure this matches exactly in FMOD
(case matters)
    [Range(0f, 1f)]
    [SerializeField] private float musicVolume = 1f;

    private VCA sfxVca;
    private bool sfxVcaValid;

    private VCA musicVca;
    private bool musicVcaValid;

    private Bus sfxBus;
    private bool sfxBusValid;

    [Header("UI")]
    public EventReference mainButton;
    public EventReference gearButton;

    [Header("Exploration")]
    public EventReference emptyTile;

    [System.Serializable]
    public struct ItemSound
    {
        public string name; //name of the item/enemy, has to be exactly as the Description
in ScriptableObject
        public EventReference sound; //find correct sound effect from FMOD library
        [Min(1)] public int maxInPattern; //how many of that item is in a pattern (e.g.
stick = 4)
    }

    [SerializeField] private List<ItemSound> Patterns = new List<ItemSound>();

    [SerializeField] private EventReference sweetnessEvent;
    [SerializeField] private string sweetnessParamName = "Sweetness"; //this is the global
parameter in FMOD and is case sensitive, has to be exactly as there is
    [SerializeField] private int sweetnessSteps = 5; //how many different sweetness sounds
there is, 0 is first so 0-4 = 5

    private readonly Dictionary<string, ItemSound> itemById = new Dictionary<string, Item-
Sound>();

    private string lastItemId = null;
    private int lastMaxCount = 1;
```

```
    #region Set Up
    private void Awake()
    {
        if (Instance != null && Instance != this)
        {
            Destroy(gameObject);
            return;
        }

        Instance = this;
        DontDestroyOnLoad(gameObject);

        // Build item lookup
        itemById.Clear();
        if (Patterns != null)
        {
            foreach (var it in Patterns)
            {
                if (string.IsNullOrWhiteSpace(it.name)) continue;

                if (itemById.ContainsKey(it.name))
                    Debug.LogWarning($"[FMODAudioManager] Duplicate item id in Items list:
'{it.name}' (later entry overwrites earlier).");

                itemById[it.name] = it;
            }
        }

        // get SFX VCA from FMOD to control sfx volume
        sfxVcaValid = false;
        if (!string.IsNullOrEmpty(sfxVcaPath))
        {
            try
            {
                sfxVca = RuntimeManager.GetVCA(sfxVcaPath);
                sfxVcaValid = sfxVca.isValid();
                Debug.Log($"[FMODAudioManager] VCA '{sfxVcaPath}' valid={sfxVcaValid}");
            }
            catch (System.Exception e)
            {
                Debug.LogWarning($"[FMODAudioManager] VCA lookup failed: {sfxVcaPath}.
({e.Message})");
            }
        }

        // get music VCA from FMOD to control music volume
        musicVcaValid = false;
        if (!string.IsNullOrEmpty(musicVcaPath))
        {
            try
            {
                musicVca = RuntimeManager.GetVCA(musicVcaPath);
                musicVcaValid = musicVca.isValid();
                Debug.Log($"[FMODAudioManager] VCA '{musicVcaPath}' valid={musicVcaVa-
lid}");
            }
            catch (System.Exception e)
            {
                Debug.LogWarning($"[FMODAudioManager] VCA lookup failed: {musicVcaPath}.
({e.Message})");
            }
        }
    }
    #endregion
```

```
    #region Volume Settings
    private void Start()
    {
        ApplyVolumes();
    }

    public void SetSfxVolume(float v)
    {
        sfxVolume = Mathf.Clamp01(v); ApplyVolumes();
    }
    public void SetMusicVolume(float v)
    {
        musicVolume = Mathf.Clamp01(v); ApplyVolumes();
    }

#if UNITY_EDITOR
    private void OnValidate()
    {
        if (!Application.isPlaying) return;
        ApplyVolumes();
    }
#endif

    private void ApplyVolumes()
    {
        ApplyOne(sfxVca, sfxVcaValid, sfxVcaPath, sfxVolume);
        ApplyOne(musicVca, musicVcaValid, musicVcaPath, musicVolume);
    }

    private static void ApplyOne(VCA vca, bool valid, string path, float volume01)
    {
        vca.setVolume(Mathf.Clamp01(volume01));
    }

    #endregion
```

```
#region Exploration Pattern (Item/Enemy) Sound
    //reason this is different method from SingleSound is that reference is from datacon-
tainer.dataname string of text and works with that method as is, without fiddling around
    public void PlayPatternSound(string itemName)
    {
        if (string.IsNullOrWhiteSpace(itemName))
        {
            Debug.LogWarning("[FMODAudioManager] Tried to play item sound with empty
name.");
            return;
        }

        if (!itemById.TryGetValue(itemName, out var it))
        {
            Debug.LogWarning($"[FMODAudioManager] Unknown item sound id: '{itemName}'.
Known ids: {string.Join(", ", itemById.Keys)}");
            return;
        }

        lastItemId = itemName;
        lastMaxCount = Mathf.Max(1, it.maxInPattern);

        if (it.sound.IsNull)
        {
            Debug.LogWarning($"[FMODAudioManager] Item '{itemName}' has null/invalid Even-
tReference.");
            return;
        }

        RuntimeManager.PlayOneShot(it.sound);
    }
    #endregion

    #region Sweetness Sound
    public void PlaySweetnessSounds(int sequenceCount)
    {
        if (string.IsNullOrEmpty(lastItemId))
        {
            Debug.LogWarning("[FMODAudioManager] PlaySweetnessSounds called before any
PlayItemSound (max unknown).");
            return;
        }

        if (sweetnessEvent.IsNull)
        {
            Debug.LogWarning("[FMODAudioManager] Sweetness event is null/invalid.");
            return;
        }

        int max = Mathf.Max(1, lastMaxCount);
        int clamped = Mathf.Clamp(sequenceCount, 0, max - 1);

        int steps = Mathf.Max(2, sweetnessSteps);
        int maxSweetIndex = steps - 1;

        int sweetIndex = (max == 1)
            ? maxSweetIndex
            : Mathf.RoundToInt((clamped / (float)(max - 1)) * maxSweetIndex);

        RuntimeManager.StudioSystem.setParameterByName(sweetnessParamName, sweetIndex);
        RuntimeManager.PlayOneShot(sweetnessEvent);
    }
    #endregion
```

```
    #region Singe Sound (e.g. UI)

    public void PlaySingleSound(EventReference sound)
    {
        if (sound.IsNull)
        {
            Debug.LogWarning("[FMODAudioManager] Tried to play null single sound.");
            return;
        }
        RuntimeManager.PlayOneShot(sound);
    }
}
#endregion
```